SAT-Based Hardware Verification

Circuit encoding, equivalence, and bounded model checking

Basel Kelziye & David Morilla

Universitat Politècnica de Catalunya



Why Check Hardware Equivalence?

1. Check synthesis optimizations

Did synthesis preserve the design's behavior?

Compare the original RTL with the synthesized, optimized netlist.

Why Check Hardware Equivalence?

1. Check synthesis optimizations

Did synthesis preserve the design's behavior?

Compare the original RTL with the synthesized, optimized netlist.

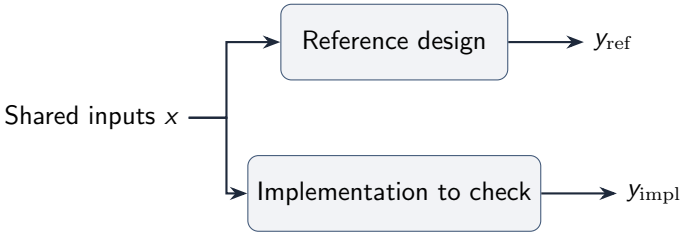
2. Compare two RTL implementations

Do two different implementations have the same behavior?

Compare RTL design A with RTL design B, even if their structures differ.

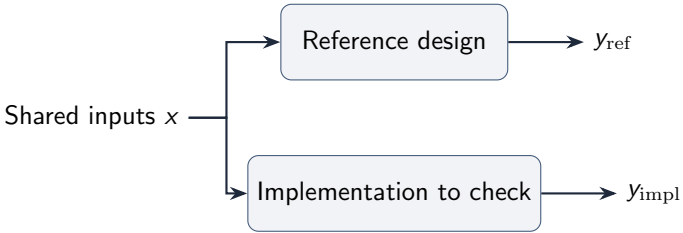
For combinational designs: same outputs for every input combination.

Search for a Disagreement



Equivalent: $\forall x,\ y_{\text{ref}}(x) = y_{\text{impl}}(x)$

Search for a Disagreement



Equivalent: $\forall x,\ y_{\mathrm{ref}}(x) = y_{\mathrm{impl}}(x)$

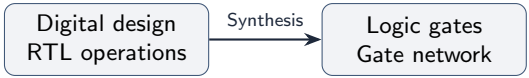
Search instead: $\exists x,\ y_{\mathrm{ref}}(x) \neq y_{\mathrm{impl}}(x)$

One input producing different outputs is enough to disprove equivalence.

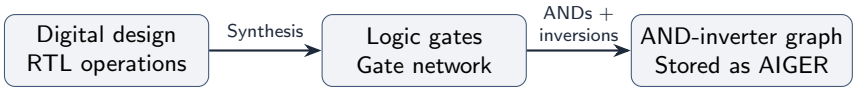
From a Digital Design to a SAT Question

Digital design
RTL operations

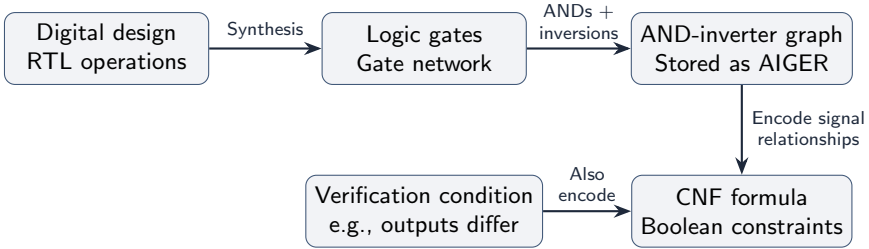
From a Digital Design to a SAT Question



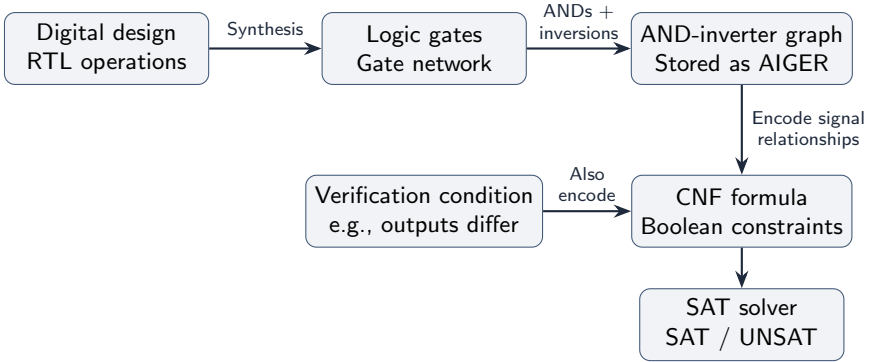
From a Digital Design to a SAT Question



From a Digital Design to a SAT Question



From a Digital Design to a SAT Question



Circuit constraints + verification condition = the question we solve.



SAT

What Is SAT?

Can Boolean variables be assigned so that a formula becomes true?

- **SAT**: at least one satisfying assignment exists.
- **UNSAT**: no satisfying assignment exists; e.g., $a \wedge \neg a$.

Searching with binary decisions

Try a variable at 0 or 1. If a branch leads to a conflict, backtrack and try the other value.

The Solver's Language: CNF

$$F = \underbrace{(\neg a \vee b \vee c)}_{C_1} \wedge \underbrace{(a \vee c \vee d)}_{C_2} \wedge \cdots \wedge C_8$$

- A **literal** is a variable or its negation: a, $\neg a$.

The example that follows

Four Boolean variables: a, b, c, d.

Eight clauses: $F = C_1 \wedge C_2 \wedge \cdots \wedge C_8$.

Every clause must be true; each needs at least one true literal.

The Solver's Language: CNF

$$F = \underbrace{(\neg a \vee b \vee c)}_{C_1} \wedge \underbrace{(a \vee c \vee d)}_{C_2} \wedge \cdots \wedge C_8$$

- A **literal** is a variable or its negation: a, $\neg a$.
- A **clause** is an OR of literals.

The example that follows

Four Boolean variables: a, b, c, d.

Eight clauses: $F = C_1 \wedge C_2 \wedge \cdots \wedge C_8$.

Every clause must be true; each needs at least one true literal.

The Solver's Language: CNF

$$F = \underbrace{(\neg a \vee b \vee c)}_{C_1} \wedge \underbrace{(a \vee c \vee d)}_{C_2} \wedge \cdots \wedge C_8$$

- A **literal** is a variable or its negation: a, $\neg a$.
- A **clause** is an OR of literals.
- **Conjunctive normal form (CNF)** is an AND of clauses.

The example that follows

Four Boolean variables: a, b, c, d.

Eight clauses: $F = C_1 \wedge C_2 \wedge \cdots \wedge C_8$.

Every clause must be true; each needs at least one true literal.



DIMACS

DIMACS: Writing CNF as a Text File

Give each Boolean variable a positive integer

$$a \mapsto 1, \qquad b \mapsto 2, \qquad c \mapsto 3, \qquad d \mapsto 4$$

DIMACS: Writing CNF as a Text File

Give each Boolean variable a positive integer

$$a \mapsto 1, \qquad b \mapsto 2, \qquad c \mapsto 3, \qquad d \mapsto 4$$

Write each clause using signed literals

$$(\neg a \vee b \vee c) \quad \longrightarrow \quad \texttt{-1 2 3 0}$$

A positive integer means the variable; a negative integer means its negation.

`0` ends a clause. It is not a Boolean variable or a false literal.

DIMACS: Writing CNF as a Text File

Give each Boolean variable a positive integer

$$a \mapsto 1, \qquad b \mapsto 2, \qquad c \mapsto 3, \qquad d \mapsto 4$$

Write each clause using signed literals

$$(\neg a \vee b \vee c) \quad \longrightarrow \quad \texttt{-1 2 3 0}$$

A positive integer means the variable; a negative integer means its negation.

`0` ends a clause. It is not a Boolean variable or a false literal.

Add a header and optional comments

`p cnf 4 8`: four variables, eight clauses.

Lines starting with `c` are comments.

DIMACS stores the same CNF formula; all its clauses must hold.

Our Eight Clauses in DIMACS

The formula from our SAT example

$C_1 \quad \neg a \vee b \vee c$
$C_2 \quad a \vee c \vee d$
$C_3 \quad a \vee c \vee \neg d$
$C_4 \quad a \vee \neg c \vee d$
$C_5 \quad a \vee \neg c \vee \neg d$
$C_6 \quad \neg b \vee \neg c \vee d$
$C_7 \quad \neg a \vee b \vee \neg c$
$C_8 \quad \neg a \vee \neg b \vee c$

$F = C_1 \wedge C_2 \wedge \cdots \wedge C_8$

Our Eight Clauses in DIMACS

The formula from our SAT example

C_1 $\neg a \vee b \vee c$
C_2 $a \vee c \vee d$
C_3 $a \vee c \vee \neg d$
C_4 $a \vee \neg c \vee d$
C_5 $a \vee \neg c \vee \neg d$
C_6 $\neg b \vee \neg c \vee d$
C_7 $\neg a \vee b \vee \neg c$
C_8 $\neg a \vee \neg b \vee c$

$F = C_1 \wedge C_2 \wedge \cdots \wedge C_8$

example.cnf

```
c a=1 b=2 c=3 d=4
p cnf 4 8
-1 2 3 0
1 3 4 0
1 3 -4 0
1 -3 4 0
1 -3 -4 0
-2 -3 4 0
-1 2 -3 0
-1 -2 3 0
```

Our Eight Clauses in DIMACS

The formula from our SAT example

C_1 $\neg a \vee b \vee c$
C_2 $a \vee c \vee d$
C_3 $a \vee c \vee \neg d$
C_4 $a \vee \neg c \vee d$
C_5 $a \vee \neg c \vee \neg d$
C_6 $\neg b \vee \neg c \vee d$
C_7 $\neg a \vee b \vee \neg c$
C_8 $\neg a \vee \neg b \vee c$

$F = C_1 \wedge C_2 \wedge \cdots \wedge C_8$

example.cnf

```
c a=1 b=2 c=3 d=4
p cnf 4 8
-1 2 3 0
1 3 4 0
1 3 -4 0
1 -3 4 0
1 -3 -4 0
-2 -3 4 0
-1 2 -3 0
-1 -2 3 0
```

Each row is one clause. The solver searches for values of variables 1–4 that satisfy all eight rows.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \ \wedge$$

$$(a \vee c \vee d) \ \wedge$$

$$(a \vee c \vee \neg d) \ \wedge$$

$$(a \vee \neg c \vee d) \ \wedge$$

$$(a \vee \neg c \vee \neg d) \ \wedge$$

$$(\neg b \vee \neg c \vee d) \ \wedge$$

$$(\neg a \vee b \vee \neg c) \ \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Each edge assigns 0 or 1 to the variable at its parent node.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

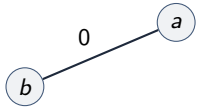
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Decide $a = 0$. Clauses containing $\neg a$ are satisfied.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

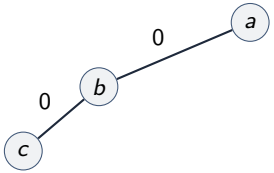
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Decide $b = 0$. Next, branch on c.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

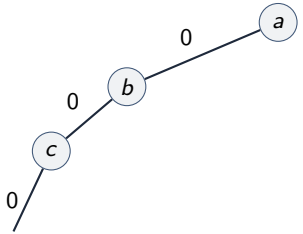
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Try $c = 0$. Simplify the remaining clauses.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

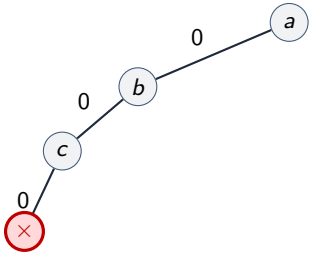
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Conflict: $(a \vee c \vee d)$ requires $d = 1$, but $(a \vee c \vee \neg d)$ requires $d = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

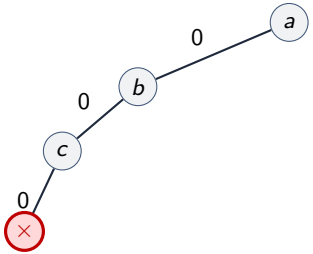
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Backtrack: undo $c = 0$ and try $c = 1$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

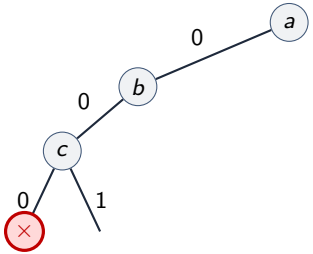
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Try $c = 1$. Simplify the remaining clauses.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

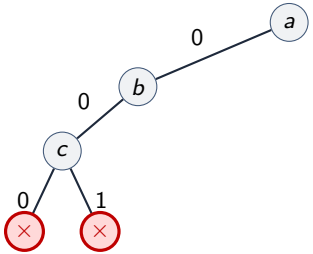
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Conflict: $(a \vee \neg c \vee d)$ requires $d = 1$, but $(a \vee \neg c \vee \neg d)$ requires $d = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

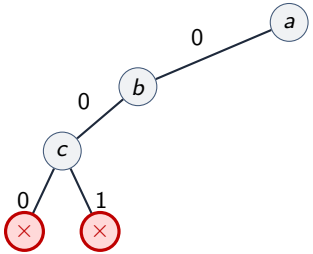
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Backtrack: both values of c failed; undo $b = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

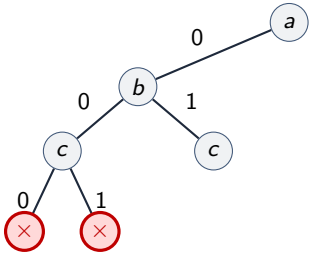
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Try $b = 1$ while keeping $a = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

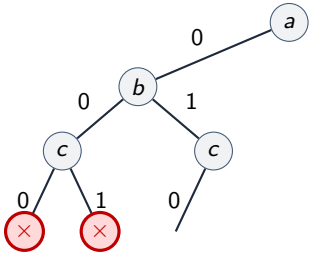
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Try $c = 0$ on this branch.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

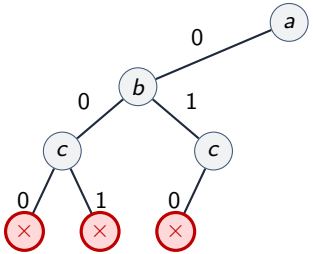
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Conflict: $(a \vee c \vee d)$ requires $d = 1$, but $(a \vee c \vee \neg d)$ requires $d = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

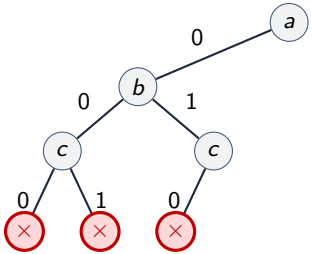
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Backtrack: undo $c = 0$ and try $c = 1$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

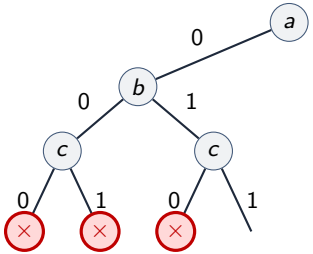
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Try $c = 1$. Simplify the remaining clauses.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

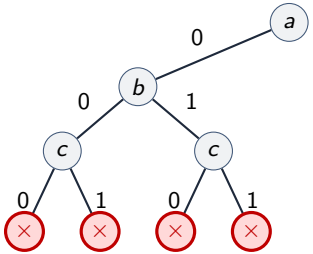
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Conflict: $(a \vee \neg c \vee d)$ requires $d = 1$, but $(a \vee \neg c \vee \neg d)$ requires $d = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \ \wedge$$

$$(a \vee c \vee d) \ \wedge$$

$$(a \vee c \vee \neg d) \ \wedge$$

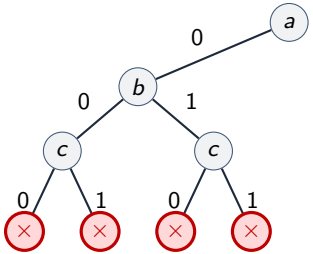
$$(a \vee \neg c \vee d) \ \wedge$$

$$(a \vee \neg c \vee \neg d) \ \wedge$$

$$(\neg b \vee \neg c \vee d) \ \wedge$$

$$(\neg a \vee b \vee \neg c) \ \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Backtrack: both values of b failed; undo $a = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \ \wedge$$

$$(a \vee c \vee d) \ \wedge$$

$$(a \vee c \vee \neg d) \ \wedge$$

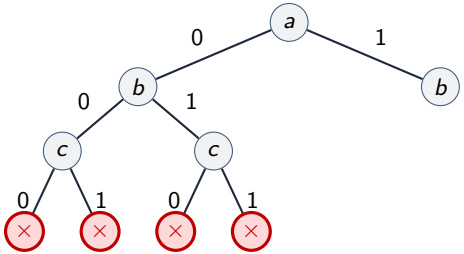
$$(a \vee \neg c \vee d) \ \wedge$$

$$(a \vee \neg c \vee \neg d) \ \wedge$$

$$(\neg b \vee \neg c \vee d) \ \wedge$$

$$(\neg a \vee b \vee \neg c) \ \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Try $a = 1$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \ \wedge$$

$$(a \vee c \vee d) \ \wedge$$

$$(a \vee c \vee \neg d) \ \wedge$$

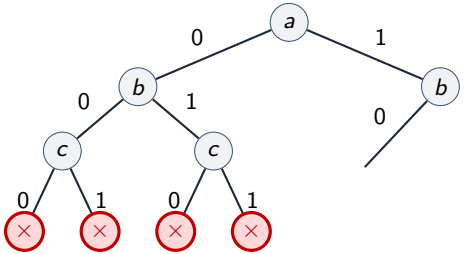
$$(a \vee \neg c \vee d) \ \wedge$$

$$(a \vee \neg c \vee \neg d) \ \wedge$$

$$(\neg b \vee \neg c \vee d) \ \wedge$$

$$(\neg a \vee b \vee \neg c) \ \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Try $b = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \ \wedge$$

$$(a \vee c \vee d) \ \wedge$$

$$(a \vee c \vee \neg d) \ \wedge$$

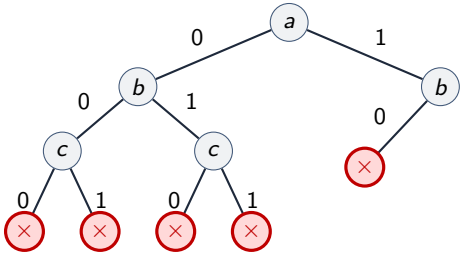
$$(a \vee \neg c \vee d) \ \wedge$$

$$(a \vee \neg c \vee \neg d) \ \wedge$$

$$(\neg b \vee \neg c \vee d) \ \wedge$$

$$(\neg a \vee b \vee \neg c) \ \wedge$$

$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Conflict: $(\neg a \vee b \vee c)$ requires $c = 1$, but $(\neg a \vee b \vee \neg c)$ requires $c = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \ \wedge$$

$$(a \vee c \vee d) \ \wedge$$

$$(a \vee c \vee \neg d) \ \wedge$$

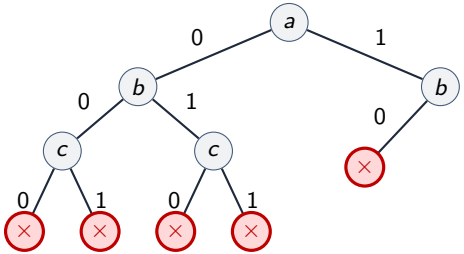
$$(a \vee \neg c \vee d) \ \wedge$$

$$(a \vee \neg c \vee \neg d) \ \wedge$$

$$(\neg b \vee \neg c \vee d) \ \wedge$$

$$(\neg a \vee b \vee \neg c) \ \wedge$$

$$(\neg a \vee \neg b \vee c)$$



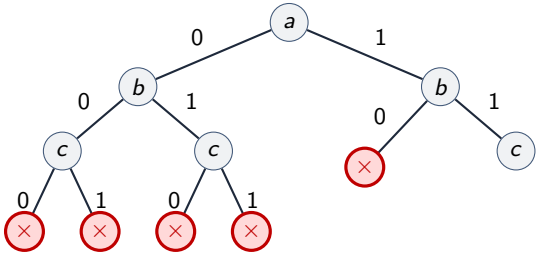
Green = satisfied by the current decisions.

Backtrack: undo $b = 0$ and try $b = 1$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$
$$(a \vee c \vee d) \wedge$$
$$(a \vee c \vee \neg d) \wedge$$
$$(a \vee \neg c \vee d) \wedge$$
$$(a \vee \neg c \vee \neg d) \wedge$$
$$(\neg b \vee \neg c \vee d) \wedge$$
$$(\neg a \vee b \vee \neg c) \wedge$$
$$(\neg a \vee \neg b \vee c)$$



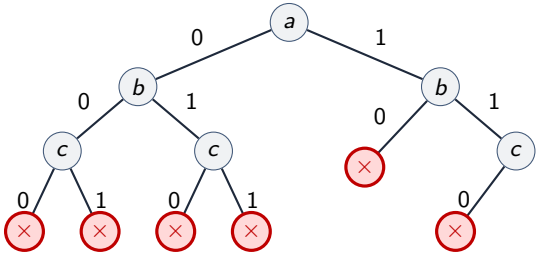
Green = satisfied by the current decisions.

Try $b = 1$ while keeping $a = 1$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$
$$(a \vee c \vee d) \wedge$$
$$(a \vee c \vee \neg d) \wedge$$
$$(a \vee \neg c \vee d) \wedge$$
$$(a \vee \neg c \vee \neg d) \wedge$$
$$(\neg b \vee \neg c \vee d) \wedge$$
$$(\neg a \vee b \vee \neg c) \wedge$$
$$(\neg a \vee \neg b \vee c)$$



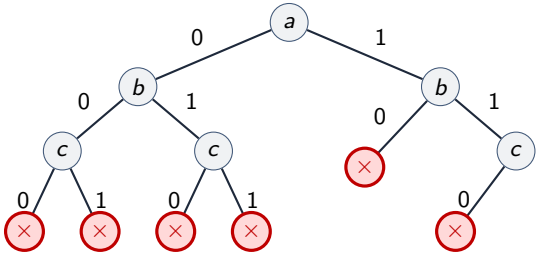
Green = satisfied by the current decisions.

Try $c = 0$: **conflict**, because $(\neg a \vee \neg b \vee c) = 0$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$
$$(a \vee c \vee d) \wedge$$
$$(a \vee c \vee \neg d) \wedge$$
$$(a \vee \neg c \vee d) \wedge$$
$$(a \vee \neg c \vee \neg d) \wedge$$
$$(\neg b \vee \neg c \vee d) \wedge$$
$$(\neg a \vee b \vee \neg c) \wedge$$
$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

Backtrack: undo $c = 0$ and try $c = 1$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$

$$(a \vee c \vee d) \wedge$$

$$(a \vee c \vee \neg d) \wedge$$

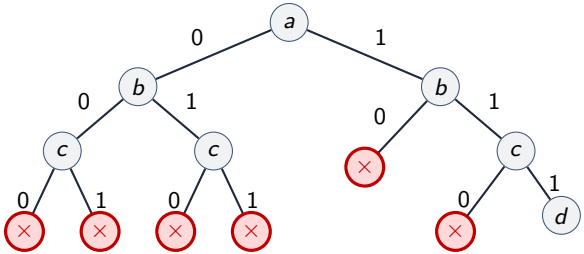
$$(a \vee \neg c \vee d) \wedge$$

$$(a \vee \neg c \vee \neg d) \wedge$$

$$(\neg b \vee \neg c \vee d) \wedge$$

$$(\neg a \vee b \vee \neg c) \wedge$$

$$(\neg a \vee \neg b \vee c)$$



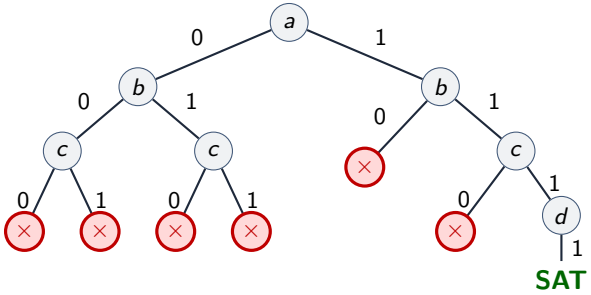
Green = satisfied by the current decisions.

Try $c = 1$. The remaining clause $(\neg b \vee \neg c \vee d)$ requires $d = 1$.

SAT Example: Binary Decisions and Backtracking

Require all eight clauses:

$$(\neg a \vee b \vee c) \wedge$$
$$(a \vee c \vee d) \wedge$$
$$(a \vee c \vee \neg d) \wedge$$
$$(a \vee \neg c \vee d) \wedge$$
$$(a \vee \neg c \vee \neg d) \wedge$$
$$(\neg b \vee \neg c \vee d) \wedge$$
$$(\neg a \vee b \vee \neg c) \wedge$$
$$(\neg a \vee \neg b \vee c)$$



Green = satisfied by the current decisions.

SAT: $a = b = c = d = 1$ satisfies all eight clauses.



AIGER

The AIGER File Format

AIGER stores an AND-inverter graph: two-input ANDs and inversions. We use its readable ASCII form (`.aag`).

Basic header: `aag M I L O A`

`M`	Maximum variable index
`I`	Number of inputs
`L`	Number of latches (state elements)
`O`	Number of outputs
`A`	Number of AND gates

This is the basic circuit format; extensions can add property fields.

The AIGER File Format

AIGER stores an AND-inverter graph: two-input ANDs and inversions. We use its readable ASCII form (`.aag`).

Basic header: `aag M I L O A`

`M`	Maximum variable index
`I`	Number of inputs
`L`	Number of latches (state elements)
`O`	Number of outputs
`A`	Number of AND gates

What follows the header, in order?

`I` input lines → `L` latch lines → `O` output lines → `A` AND lines.

Optional signal names and comments may follow.

This is the basic circuit format; extensions can add property fields.

Reading Signals and Gates

Signals are written as integer literals

For variable index $v \geq 1$:

$$2v \text{ means the signal,} \qquad 2v + 1 \text{ means its negation.}$$

Even literals are unnegated; odd literals are inverted.

The special literals `0` and `1` mean false and true.

Reference: AIGER format specification, Armin Biere.

Reading Signals and Gates

Signals are written as integer literals

For variable index $v \geq 1$:

$2v$ means the signal, $2v + 1$ means its negation.

Even literals are unnegated; odd literals are inverted.

The special literals `0` and `1` mean false and true.

Line contents in the basic ASCII format

Input	One literal
Latch	Current-state literal, next-state literal
Output	One literal (possibly inverted)
AND	Output literal, first input literal, second input literal

Reference: AIGER format specification, Armin Biere.

Reading Signals and Gates

Signals are written as integer literals

For variable index $v \geq 1$:

$$2v \text{ means the signal,} \qquad 2v + 1 \text{ means its negation.}$$

Even literals are unnegated; odd literals are inverted.

The special literals `0` and `1` mean false and true.

Line contents in the basic ASCII format

Input	One literal
Latch	Current-state literal, next-state literal
Output	One literal (possibly inverted)
AND	Output literal, first input literal, second input literal

An inversion changes a literal's parity; it needs no separate gate line.

Reference: AIGER format specification, Armin Biere.

Why Convert AIGER to CNF?

The circuit and the solver use different representations

AIGER describes how signals are connected.

CNF-based SAT solvers accept Boolean constraints in **conjunctive normal form (CNF)**: an AND of clauses, where each clause is an OR of literals.

Why Convert AIGER to CNF?

The circuit and the solver use different representations

AIGER describes how signals are connected.

CNF-based SAT solvers accept Boolean constraints in **conjunctive normal form (CNF)**: an AND of clauses, where each clause is an OR of literals.

Preserve the behavior as constraints

Assign Boolean variables to signals and constrain each gate's output to match its inputs. A satisfying assignment then describes a legal combination of circuit signal values.

Why Convert AIGER to CNF?

The circuit and the solver use different representations

AIGER describes how signals are connected.

CNF-based SAT solvers accept Boolean constraints in **conjunctive normal form (CNF)**: an AND of clauses, where each clause is an OR of literals.

Preserve the behavior as constraints

Assign Boolean variables to signals and constrain each gate's output to match its inputs. A satisfying assignment then describes a legal combination of circuit signal values.

Encode the circuit first; later, add the miter's requirement that outputs disagree.

A SAT Variable Names a Signal Value



Three wires, three Boolean values

a and b name the input values; n names the output value.

Introducing n gives the existing output wire a name in the formula.

A SAT Variable Names a Signal Value



Three wires, three Boolean values

a and b name the input values; n names the output value.

Introducing n gives the existing output wire a name in the formula.

The gate constrains which values can occur together

$$n \leftrightarrow (a \wedge b)$$

a	b	n	
1	0	0	Legal: the gate's output is 0.
1	0	1	Illegal: the constraints must reject this.
1	1	1	Legal: both inputs are 1.

A SAT Variable Names a Signal Value



Three wires, three Boolean values

a and b name the input values; n names the output value.

Introducing n gives the existing output wire a name in the formula.

The gate constrains which values can occur together

$$n \leftrightarrow (a \wedge b)$$

a	b	n	
1	0	0	Legal: the gate's output is 0.
1	0	1	Illegal: the constraints must reject this.
1	1	1	Legal: both inputs are 1.

The solver assigns signal values; the clauses enforce the gate's behavior.

How the Conversion Works

Encode a two-input AND: $n \leftrightarrow (b \wedge a)$

How the Conversion Works

Encode a two-input AND: $n \leftrightarrow (b \wedge a)$

If $n = 1$, both inputs must be 1:

$$(\neg n \vee b) \wedge (\neg n \vee a)$$

How the Conversion Works

Encode a two-input AND: $n \leftrightarrow (b \wedge a)$

If $n = 1$, both inputs must be 1:

$$(\neg n \vee b) \wedge (\neg n \vee a)$$

If both inputs are 1, n must be 1:

$$(n \vee \neg b \vee \neg a)$$

How the Conversion Works

Encode a two-input AND: $n \leftrightarrow (b \wedge a)$

If $n = 1$, both inputs must be 1:

$$(\neg n \vee b) \wedge (\neg n \vee a)$$

If both inputs are 1, n must be 1:

$$(n \vee \neg b \vee \neg a)$$

Repeat for each AND node and combine all clauses. Reuse the same variable for a signal wherever it connects gates. Inversions become negated literals. This is **Tseitin encoding**: three clauses per AND, so the encoding grows linearly with the graph.

How the Conversion Works

Encode a two-input AND: $n \leftrightarrow (b \wedge a)$

If $n = 1$, both inputs must be 1:

$$(\neg n \vee b) \wedge (\neg n \vee a)$$

If both inputs are 1, n must be 1:

$$(n \vee \neg b \vee \neg a)$$

Repeat for each AND node and combine all clauses. Reuse the same variable for a signal wherever it connects gates. Inversions become negated literals. This is **Tseitin encoding**: three clauses per AND, so the encoding grows linearly with the graph.

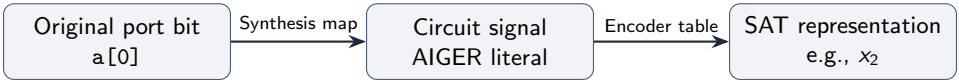
Where does DIMACS fit?

DIMACS is the text format used to write CNF for the solver. For example, if n is variable 5 and b is variable 3:

$$(\neg n \vee b) \quad \longrightarrow \quad \texttt{-5 3 0}$$

A minus sign means negation; 0 ends the clause.

Keeping Signal Names Through the Encoding



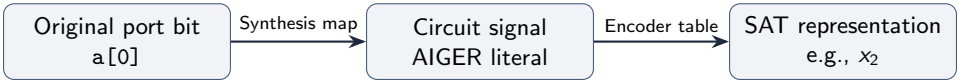
Preserve the connection between a name and a number

A bus is mapped bit by bit: `a[0]`, `a[1]`, and so on.

The synthesis map identifies which circuit input/output belongs to each port bit. The encoder then records the SAT literal representing that signal.

SAT IDs are assigned by the encoder. They are not port bit numbers or AIGER literals.

Keeping Signal Names Through the Encoding



Preserve the connection between a name and a number

A bus is mapped bit by bit: `a[0]`, `a[1]`, and so on.

The synthesis map identifies which circuit input/output belongs to each port bit. The encoder then records the SAT literal representing that signal.

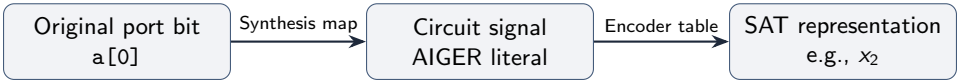
The two maps in this project

`design.map`: circuit input/output position $\rightarrow$ port name and bit.

`design.iomap`: port name and bit $\rightarrow$ signed SAT literal.

SAT IDs are assigned by the encoder. They are not port bit numbers or AIGER literals.

Keeping Signal Names Through the Encoding



Preserve the connection between a name and a number

A bus is mapped bit by bit: `a[0]`, `a[1]`, and so on.

The synthesis map identifies which circuit input/output belongs to each port bit. The encoder then records the SAT literal representing that signal.

The two maps in this project

`design.map`: circuit input/output position → port name and bit.

`design.iomap`: port name and bit → signed SAT literal.

The CNF stores constraints; the I/O map preserves the names needed to interpret them.

SAT IDs are assigned by the encoder. They are not port bit numbers or AIGER literals.

Reading an I/O Map

Each record: direction, port, bit index, variable ID, polarity

Direction	Port	Bit	Variable	Polarity	Meaning
`i`	`a`	0	2	+	$a[0] = x_2$
`i`	`a`	1	3	+	$a[1] = x_3$
`o`	`y`	0	5	−	$y = \neg x_5$

Illustrative IDs for a two-bit input and a one-bit output.

Reading an I/O Map

Each record: direction, port, bit index, variable ID, polarity

Direction	Port	Bit	Variable	Polarity	Meaning
i	a	0	2	+	$a[0] = x_2$
i	a	1	3	+	$a[1] = x_3$
o	y	0	5	−	$y = \neg x_5$

Illustrative IDs for a two-bit input and a one-bit output.

Read a mapped signal using its polarity

If the model gives $x_5 = 1$, the mapped output $y = \neg x_5$ is $\mathbf{0}$.

An output can reference an existing input or gate literal; it does not always need a new variable. Inversion is represented by the sign.

Reading an I/O Map

Each record: direction, port, bit index, variable ID, polarity

Direction	Port	Bit	Variable	Polarity	Meaning
i	a	0	2	+	$a[0] = x_2$
i	a	1	3	+	$a[1] = x_3$
o	y	0	5	−	$y = \neg x_5$

Illustrative IDs for a two-bit input and a one-bit output.

Read a mapped signal using its polarity

If the model gives $x_5 = 1$, the mapped output $y = \neg x_5$ is **0**.

An output can reference an existing input or gate literal; it does not always need a new variable. Inversion is represented by the sign.

The map also tells the miter which signals correspond

Match inputs and outputs by **port name and bit index**, even if their SAT IDs differ between the circuits.



Miter Circuit

What Is a Miter Circuit?

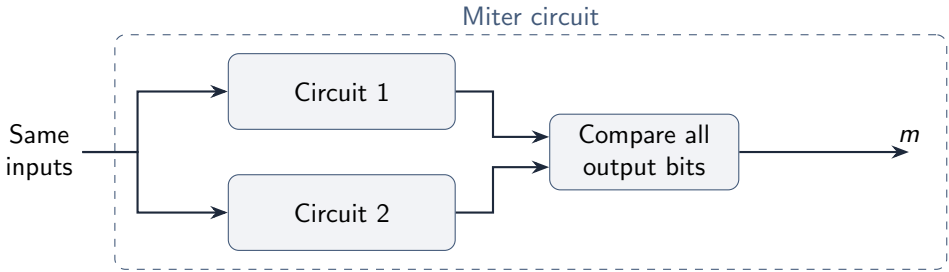
Definition

A miter combines two circuits with matching inputs and compares their outputs. Its output is 1 whenever at least one corresponding output bit differs.

What Is a Miter Circuit?

Definition

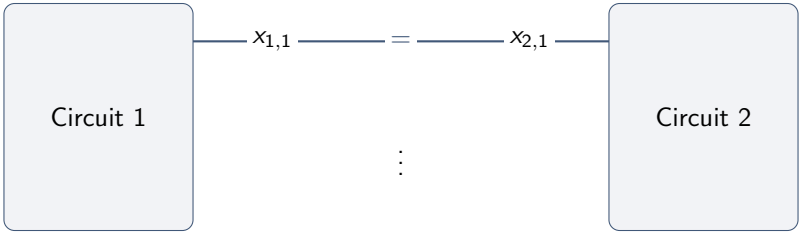
A miter combines two circuits with matching inputs and compares their outputs. Its output is 1 whenever at least one corresponding output bit differs.



Ask the SAT solver: can this miter produce $m = 1$?

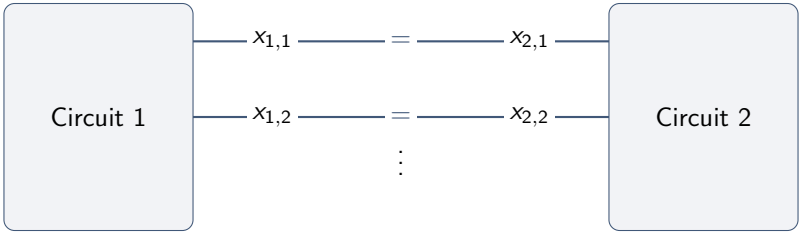
Bind Corresponding Inputs

Both circuits must receive the same input values. Establish which input bits correspond before comparing their outputs.



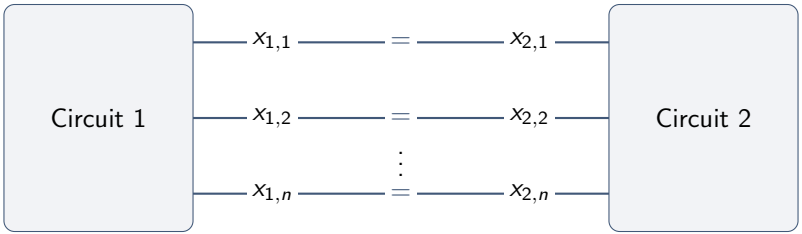
Bind Corresponding Inputs

Both circuits must receive the same input values. Establish which input bits correspond before comparing their outputs.



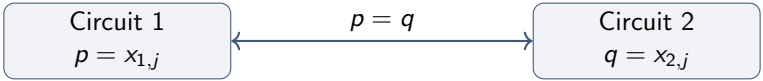
Bind Corresponding Inputs

Both circuits must receive the same input values. Establish which input bits correspond before comparing their outputs.

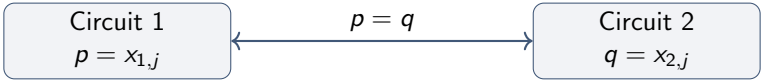


For each input bit j: $x_{1,j} \leftrightarrow x_{2,j}$. Separate SAT variables can be constrained to have equal values.

One Input Binding, Two Implications



One Input Binding, Two Implications



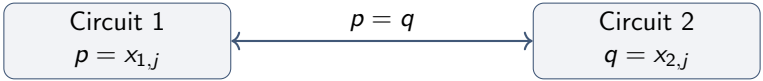
Both implications are required

$$p \Rightarrow q \quad \equiv \quad (\neg p \vee q)$$

$$q \Rightarrow p \quad \equiv \quad (\neg q \vee p)$$

Together, these allow only $(p, q) = (0, 0)$ or $(1, 1)$.

One Input Binding, Two Implications



Both implications are required

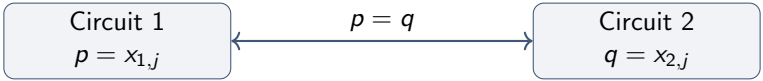
$$p \Rightarrow q \quad \equiv \quad (\neg p \vee q)$$

$$q \Rightarrow p \quad \equiv \quad (\neg q \vee p)$$

Together, these allow only $(p, q) = (0, 0)$ or $(1, 1)$.

$$\boxed{(\neg p \vee q) \wedge (p \vee \neg q)}$$

One Input Binding, Two Implications



Both implications are required

$$p \Rightarrow q \quad \equiv \quad (\neg p \vee q)$$

$$q \Rightarrow p \quad \equiv \quad (\neg q \vee p)$$

Together, these allow only $(p, q) = (0, 0)$ or $(1, 1)$.

$$\boxed{(\neg p \vee q) \wedge (p \vee \neg q)}$$

Repeat for all n input bits: $2n$ equality clauses.

Which SAT Variables Are Introduced?

Keep the two circuit encodings separate

Assign disjoint variable ranges to the two circuits. Each circuit's internal signals retain their own variables.

Which SAT Variables Are Introduced?

Keep the two circuit encodings separate

Assign disjoint variable ranges to the two circuits. Each circuit's internal signals retain their own variables.

Bind corresponding inputs

Add two equality clauses for each matching input pair. No extra variable is needed for these equality constraints.

Alternatively, both encodings can share the same primary-input variables.

Which SAT Variables Are Introduced?

Keep the two circuit encodings separate

Assign disjoint variable ranges to the two circuits. Each circuit's internal signals retain their own variables.

Bind corresponding inputs

Add two equality clauses for each matching input pair. No extra variable is needed for these equality constraints.

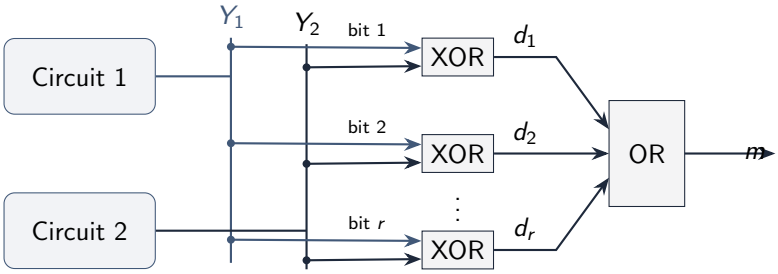
Alternatively, both encodings can share the same primary-input variables.

Add output comparison variables

For r corresponding output bits, introduce r difference variables:

$$d_j \leftrightarrow (y_{1,j} \oplus y_{2,j}), \qquad j = 1, \ldots, r.$$

Compare Every Output Bit



$$d_j = y_{1,j} \oplus y_{2,j}, \qquad m = \bigvee_{j=1}^{r} d_j$$

Any differing output bit makes $m = 1$.

Ask for at Least One Difference

Each XOR must reflect its output pair

For output literals u, v and a fresh difference variable d:

$$d \leftrightarrow (u \oplus v)$$

$$(\neg d \vee u \vee v) \wedge (\neg d \vee \neg u \vee \neg v)$$

$$\wedge (d \vee u \vee \neg v) \wedge (d \vee \neg u \vee v)$$

Ask for at Least One Difference

Each XOR must reflect its output pair

For output literals u, v and a fresh difference variable d:

$$d \leftrightarrow (u \oplus v)$$

$$(\neg d \vee u \vee v) \wedge (\neg d \vee \neg u \vee \neg v)$$

$$\wedge (d \vee u \vee \neg v) \wedge (d \vee \neg u \vee v)$$

Require at least one difference to be true

$$\boxed{(d_1 \vee d_2 \vee \cdots \vee d_r)}$$

This single clause asserts the OR directly; no separate OR variable is needed.

Ask for at Least One Difference

Each XOR must reflect its output pair

For output literals u, v and a fresh difference variable d:

$$d \leftrightarrow (u \oplus v)$$

$$(\neg d \vee u \vee v) \wedge (\neg d \vee \neg u \vee \neg v)$$

$$\wedge (d \vee u \vee \neg v) \wedge (d \vee \neg u \vee v)$$

Require at least one difference to be true

$$\boxed{(d_1 \vee d_2 \vee \cdots \vee d_r)}$$

This single clause asserts the OR directly; no separate OR variable is needed.

Both circuit encodings + input equalities + XOR constraints + mismatch clause.

Interpreting the Miter Result

SAT: a disagreement exists

A satisfying assignment gives a common input for which at least one corresponding output bit differs. Those input values form a counterexample to equivalence.

Interpreting the Miter Result

SAT: a disagreement exists

A satisfying assignment gives a common input for which at least one corresponding output bit differs. Those input values form a counterexample to equivalence.

UNSAT: no disagreement exists

No common input can satisfy the mismatch clause while respecting both circuit encodings. The outputs agree for every allowed input.

Interpreting the Miter Result

SAT: a disagreement exists

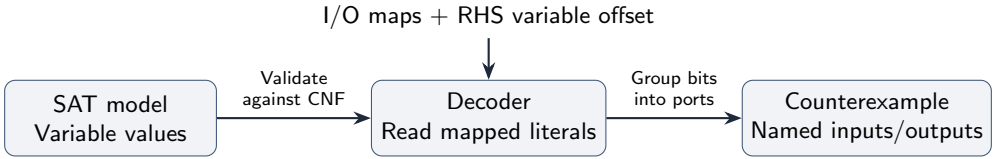
A satisfying assignment gives a common input for which at least one corresponding output bit differs. Those input values form a counterexample to equivalence.

UNSAT: no disagreement exists

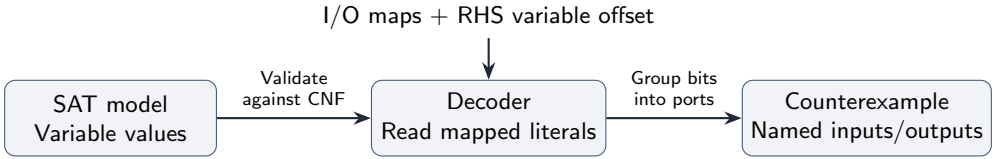
No common input can satisfy the mismatch clause while respecting both circuit encodings. The outputs agree for every allowed input.

UNSAT proves equivalence for the encoded combinational circuits under the encoded input assumptions.

From a SAT Model Back to Circuit Signals



From a SAT Model Back to Circuit Signals

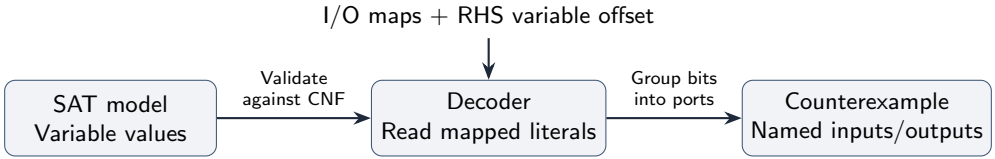


Use the same variable numbering as the miter

If the left CNF declares V_L variables, a right-side variable x_j is stored as x_{V_L+j} in the miter. Preserve the literal's sign.

Example: $V_L = 8$ and a right-side output mapped to $+x_6$ means reading $+x_{14}$ from the model.

From a SAT Model Back to Circuit Signals



Use the same variable numbering as the miter

If the left CNF declares V_L variables, a right-side variable x_j is stored as x_{V_L+j} in the miter. Preserve the literal's sign.

Example: $V_L = 8$ and a right-side output mapped to $+x_6$ means reading $+x_{14}$ from the model.

The solver finds the witness; the decoder translates it

Evaluate each mapped literal, apply its polarity, and assemble each port from its highest bit index down to its lowest. Compare corresponding outputs.

No new search is needed: the counterexample is already contained in the SAT model.

Decode a Small Counterexample by Hand

Illustrative two-bit input $a[1:0]$ and one-bit output y. The RHS variables are shifted by $V_L = 8$.

Selected values from a satisfying miter assignment

$$x_2 = 1, \quad x_3 = 0, \quad x_5 = 1, \qquad x_{10} = 1, \quad x_{11} = 0, \quad x_{14} = 1$$

These are illustrative mappings, not ALU IDs. UNSAT provides no satisfying model to decode.

Decode a Small Counterexample by Hand

Illustrative two-bit input $a[1:0]$ and one-bit output y. The RHS variables are shifted by $V_L = 8$.

Selected values from a satisfying miter assignment

$$x_2 = 1, \quad x_3 = 0, \quad x_5 = 1, \quad x_{10} = 1, \quad x_{11} = 0, \quad x_{14} = 1$$

Signal	Local I/O mapping	Miter literal	Decoded value
LHS $a[0]$	$+x_2$	$+x_2$	1
LHS $a[1]$	$+x_3$	$+x_3$	0
RHS $a[0]$	$+x_2$	$+x_{10}$	1
RHS $a[1]$	$+x_3$	$+x_{11}$	0
LHS y	$-x_5$	$-x_5$	$\neg 1 = 0$
RHS y	$+x_6$	$+x_{14}$	1

These are illustrative mappings, not ALU IDs. UNSAT provides no satisfying model to decode.

Decode a Small Counterexample by Hand

Illustrative two-bit input $a[1:0]$ and one-bit output y. The RHS variables are shifted by $V_L = 8$.

Selected values from a satisfying miter assignment

$$x_2 = 1, \quad x_3 = 0, \quad x_5 = 1, \quad x_{10} = 1, \quad x_{11} = 0, \quad x_{14} = 1$$

Signal	Local I/O mapping	Miter literal	Decoded value
LHS $a[0]$	$+x_2$	$+x_2$	1
LHS $a[1]$	$+x_3$	$+x_3$	0
RHS $a[0]$	$+x_2$	$+x_{10}$	1
RHS $a[1]$	$+x_3$	$+x_{11}$	0
LHS y	$-x_5$	$-x_5$	$\neg 1 = 0$
RHS y	$+x_6$	$+x_{14}$	1

$a = a[1]a[0] = 01_2$ on both sides, $\quad y_L = 0 \neq y_R = 1$.

The common input $a = 01_2$ is a counterexample to equivalence.

These are illustrative mappings, not ALU IDs. UNSAT provides no satisfying model to decode.



Bounded Model Checking

BMC: Checking a Property Over Time

The question

Can a sequential design violate a chosen safety property within K clock steps?

BMC: Checking a Property Over Time

The question

Can a sequential design violate a chosen safety property within K clock steps?

One design and one property

The state is the set of values stored in the registers. Inputs and circuit logic determine how that state changes.

A property describes what should hold at every step.

BMC: Checking a Property Over Time

The question

Can a sequential design violate a chosen safety property within K clock steps?

One design and one property

The state is the set of values stored in the registers. Inputs and circuit logic determine how that state changes.

A property describes what should hold at every step.

Express the failure condition

For a state-based safety property $P(s)$, define:

$$bad(s) = \neg P(s).$$

Search for a reachable state in which this failure condition is true.

Describe the Starting State and Each Step

Initial condition: $I(s_0)$

Which register values are allowed at the start?

The initial condition records the allowed starting states. Bits not constrained by it remain free.

Describe the Starting State and Each Step

Initial condition: $I(s_0)$

Which register values are allowed at the start?

The initial condition records the allowed starting states. Bits not constrained by it remain free.

Transition relation: $T(s_t, u_t, s_{t+1})$

Which next state follows from the current state and inputs?

For a deterministic circuit: $s_{t+1} = F(s_t, u_t)$.

Describe the Starting State and Each Step

Initial condition: $I(s_0)$

Which register values are allowed at the start?

The initial condition records the allowed starting states. Bits not constrained by it remain free.

Transition relation: $T(s_t, u_t, s_{t+1})$

Which next state follows from the current state and inputs?

For a deterministic circuit: $s_{t+1} = F(s_t, u_t)$.

Failure condition: $\neg P(s_t)$

The failure condition is true exactly when the safety property is false.

Describe the Starting State and Each Step

Initial condition: $I(s_0)$

Which register values are allowed at the start?

The initial condition records the allowed starting states. Bits not constrained by it remain free.

Transition relation: $T(s_t, u_t, s_{t+1})$

Which next state follows from the current state and inputs?

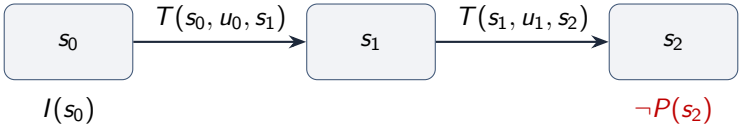
For a deterministic circuit: $s_{t+1} = F(s_t, u_t)$.

Failure condition: $\neg P(s_t)$

The failure condition is true exactly when the safety property is false.

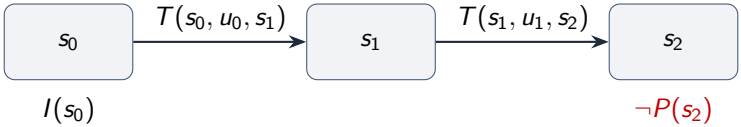
In a synchronous single-clock model, one transition represents one active clock edge.

Unroll the Design and Ask for a Failure



Each frame has distinct SAT variables. Transition constraints connect them.

Unroll the Design and Ask for a Failure

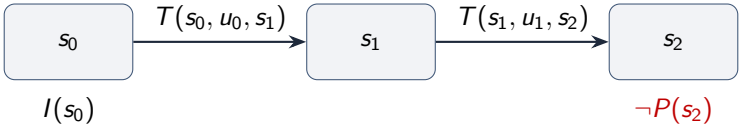


Each frame has distinct SAT variables. Transition constraints connect them.

$$B_k = I(s_0) \wedge \left(\bigwedge_{t=0}^{k-1} T(s_t, u_t, s_{t+1}) \right) \wedge \neg P(s_k)$$

The solver assigns the entire trace together; only the final frame is required to violate the property.

Unroll the Design and Ask for a Failure



Each frame has distinct SAT variables. Transition constraints connect them.

$$B_k = I(s_0) \wedge \left(\bigwedge_{t=0}^{k-1} T(s_t, u_t, s_{t+1}) \right) \wedge \neg P(s_k)$$

The solver assigns the entire trace together; only the final frame is required to violate the property.

Check depths $0, 1, \ldots, K$

SAT: a counterexample trace reaches a failure at that depth.

All UNSAT: no failure through K steps under the encoded conditions; this is not a proof for all future steps.

Unrolling: What Defines the Next State?

For latch i, let $q_{i,k}$ be its SAT variable in frame k. Q_k denotes all latch values, and u_k the inputs in that frame.

Encode the next-state logic in the previous frame

$$d_{i,k-1} \leftrightarrow F_i(Q_{k-1}, u_{k-1})$$

F_i is the circuit logic driving latch i; its gates are encoded in CNF.

Unrolling: What Defines the Next State?

For latch i, let $q_{i,k}$ be its SAT variable in frame k. Q_k denotes all latch values, and u_k the inputs in that frame.

Encode the next-state logic in the previous frame

$$d_{i,k-1} \leftrightarrow F_i(Q_{k-1}, u_{k-1})$$

F_i is the circuit logic driving latch i; its gates are encoded in CNF.

Connect that result to the latch in frame k

$$q_{i,k} \leftrightarrow d_{i,k-1} \quad \equiv \quad \boxed{(\neg q_{i,k} \vee d_{i,k-1}) \wedge (q_{i,k} \vee \neg d_{i,k-1})}$$

These two clauses force the new latch value to equal the previous frame's computed next-state value.

Unrolling: What Defines the Next State?

For latch i, let $q_{i,k}$ be its SAT variable in frame k. Q_k denotes all latch values, and u_k the inputs in that frame.

Encode the next-state logic in the previous frame

$$d_{i,k-1} \leftrightarrow F_i(Q_{k-1}, u_{k-1})$$

F_i is the circuit logic driving latch i; its gates are encoded in CNF.

Connect that result to the latch in frame k

$$q_{i,k} \leftrightarrow d_{i,k-1} \quad \equiv \quad \boxed{(\neg q_{i,k} \vee d_{i,k-1}) \wedge (q_{i,k} \vee \neg d_{i,k-1})}$$

These two clauses force the new latch value to equal the previous frame's computed next-state value.

Repeat for every latch and every $k \geq 1$. Frame 0 is constrained by initialization.

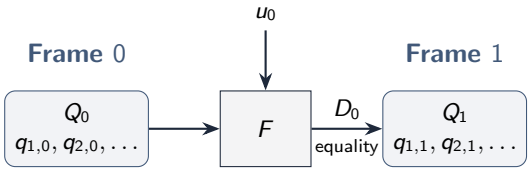
Unrolling: The Same Circuit Across Time

Frame 0

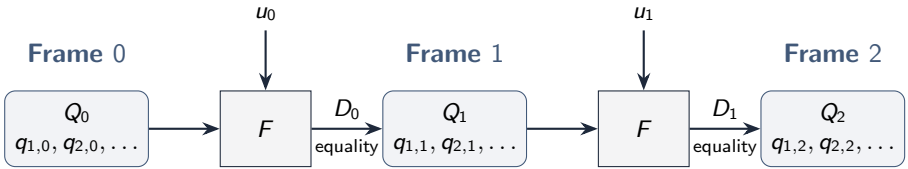
Q_0

$q_{1,0}, q_{2,0}, \ldots$

Unrolling: The Same Circuit Across Time



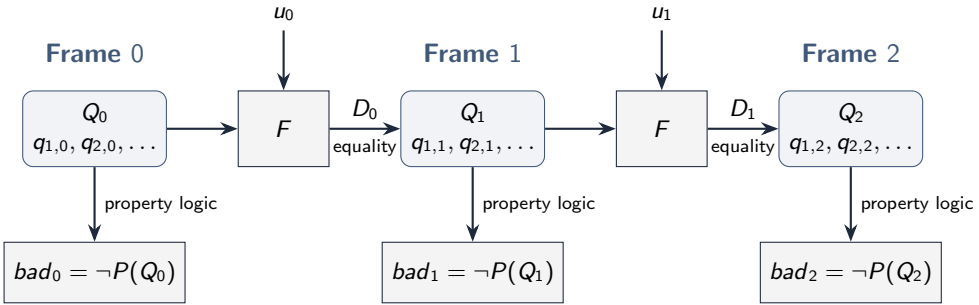
Unrolling: The Same Circuit Across Time



$$Q_1 = D_0 = F(Q_0, u_0), \qquad Q_2 = D_1 = F(Q_1, u_1).$$

Each copy uses fresh state, input, and gate variables. Equality clauses connect each bit of D_{k-1} to its bit of Q_k.

Unrolling: The Same Circuit Across Time



$$Q_1 = D_0 = F(Q_0, u_0), \qquad Q_2 = D_1 = F(Q_1, u_1).$$

Each copy uses fresh state, input, and gate variables. Equality clauses connect each bit of D_{k-1} to its bit of Q_k.

The property has a literal in each frame. At depth 2, assert only (bad_2) to search for a failure.

What LEC-SAT Implements

Our verification layer

- Encode gate-level JSON or ASCII AIGER as CNF.
- Preserve port names, bit indices, and signed literals.
- Build a combinational equivalence miter.
- Unroll sequential circuits for bounded property checks.
- Validate SAT models and decode counterexamples.

Tools we build on

Yosys: synthesize RTL and export the circuit.

MiniSat: solve the resulting CNF using CDCL.

Python + Make: connect the stages and run regression tests.

The project connects circuit semantics to a SAT query and turns a model back into hardware signals.

A separate educational backtracking solver is also available.



Walkthrough: A 4-bit ALU

Two ALUs, the same interface: $N = 4$

Inputs: $a[3:0]$, $b[3:0]$, $op[1:0]$ (10 bits).

Outputs: $result[3:0]$, $cout$ (5 bits).

Operations: addition, subtraction, bitwise AND, bitwise OR.

1. Build the Tools and Select the Designs

Two ALUs, the same interface: $N = 4$

Inputs: $a[3:0]$, $b[3:0]$, $op[1:0]$ (10 bits).

Outputs: $result[3:0]$, $cout$ (5 bits).

Operations: addition, subtraction, bitwise AND, bitwise OR.

Bash — from the `LEC-SAT/` directory

```
make tools
W="$PWD/build/walkthrough/alu"
```

The walkthrough script prepares the two folders and copies each Verilog module as `design.v`, alongside its `synth.ys` script.

Compile the verification software first; then synthesize the hardware.

The Two Verilog Modules, Side by Side

lhs.v — faulty carry

```
module lhs #(parameter integer N=4) (
  input wire [N-1:0] a, b,
  input wire [1:0] op,
  output wire [N-1:0] result,
  output wire cout
);
  wire [N:0] sum_ext = {1'b0,a} + {1'b0,b};
  wire [N:0] diff_ext = {1'b0,a} - {1'b0,b};
  reg [N-1:0] res; reg co;
  always @(*) begin case (op)
      2'b00: begin res = sum_ext[N-1:0];
        co = sum_ext[N-2];
      end
      2'b01: begin
        res = diff_ext[N-1:0]; co = diff_ext[N];
      end
      2'b10: begin res = a & b; co = 1'b0; end
      2'b11: begin res = a | b; co = 1'b0; end
  endcase end
  assign result = res; assign cout = co;
endmodule
```

rhs.v — reference

```
module rhs #(parameter integer N=4) (
  input wire [N-1:0] a, b,
  input wire [1:0] op,
  output wire [N-1:0] result,
  output wire cout
);
  wire [N:0] sum_ext = {1'b0,a} + {1'b0,b};
  wire [N:0] diff_ext = {1'b0,a} - {1'b0,b};
  reg [N-1:0] res; reg co;
  always @(*) begin case (op)
      2'b00: begin res = sum_ext[N-1:0];
        co = sum_ext[N];
      end
      2'b01: begin
        res = diff_ext[N-1:0]; co = diff_ext[N];
      end
      2'b10: begin res = a & b; co = 1'b0; end
      2'b11: begin res = a | b; co = 1'b0; end
  endcase end
  assign result = res; assign cout = co;
endmodule
```

Addition selects bit $N - 2 = 2$ on the left and carry bit $N = 4$ on the right. Other operations agree.

2. Yosys: Verilog to an AND-Inverter Graph

Yosys script: `lhs/synth.ys`

```
read_verilog design.v
chparam -set N 4 lhs
hierarchy -check -auto-top
synth
aigmap
write_aiger -ascii \
  -map design.map design.aag
```

What these steps produce

Read the copied `lhs.v`; set its width and select the top module.

Bash — run the script inside the LHS folder

```
(cd "$W/lhs" && yosys -q -s synth.ys)
```

Repeat for `rhs.v`, setting the module parameter on `rhs`.

2. Yosys: Verilog to an AND-Inverter Graph

Yosys script: `lhs/synth.ys`

```
read_verilog design.v
chparam -set N 4 lhs
hierarchy -check -auto-top
synth
aigmap
write_aiger -ascii \
  -map design.map design.aag
```

What these steps produce

Read the copied `lhs.v`; set its width and select the top module.

Synthesize and map logic to ANDs and inversions.

Bash — run the script inside the LHS folder

```
(cd "$W/lhs" && yosys -q -s synth.ys)
```

Repeat for `rhs.v`, setting the module parameter on `rhs`.

2. Yosys: Verilog to an AND-Inverter Graph

Yosys script: `lhs/synth.ys`

```
read_verilog design.v
chparam -set N 4 lhs
hierarchy -check -auto-top
synth
aigmap
write_aiger -ascii \
  -map design.map design.aag
```

What these steps produce

Read the copied `lhs.v`; set its width and select the top module.

Synthesize and map logic to ANDs and inversions.

Write `design.aag` and a port-name map.

Bash — run the script inside the LHS folder

```
(cd "$W/lhs" && yosys -q -s synth.ys)
```

Repeat for `rhs.v`, setting the module parameter on `rhs`.

Captured LHS header: `aag 111 10 0 5 101`

111 maximum variable index; 10 inputs; no latches; 5 outputs; 101 ANDs.

3. Encode Both AAG Files as CNF

```
for side in lhs rhs; do
  build/combinational/cnf_gen --from-aag \
    "$W/$side/design.aag" "$side" \
    "$W/$side/design" "$W/$side/design.map"
done
```

Two outputs per circuit

`design.cnf`: clauses enforcing the circuit's behavior.

`design.iomap`: named input/output bits mapped to SAT literals.

3. Encode Both AAG Files as CNF

```
for side in lhs rhs; do
  build/combinational/cnf_gen --from-aag \
    "$W/$side/design.aag" "$side" \
    "$W/$side/design" "$W/$side/design.map"
done
```

Two outputs per circuit

`design.cnf`: clauses enforcing the circuit's behavior.

`design.iomap`: named input/output bits mapped to SAT literals.

Circuit	SAT variables	Clauses
LHS	112	304
RHS	117	319

These formulas describe legal signal values. The comparison comes next.

One Real Gate: From AIGER to Three Clauses

The first AND line in the generated LHS AAG

22 21 19 means $g = \neg op_1 \wedge \neg op_0$.

AIGER literals 20 and 18 name op_1 and op_0; 21 and 19 invert them.

One Real Gate: From AIGER to Three Clauses

The first AND line in the generated LHS AAG

22 21 19 means $g = \neg op_1 \wedge \neg op_0$.

AIGER literals 20 and 18 name op_1 and op_0; 21 and 19 invert them.

The CNF encoder's variable names for those signals

$op_0 \mapsto x_{10}, \quad op_1 \mapsto x_{11}, \quad g \mapsto x_{12}$

$$x_{12} \leftrightarrow (\neg x_{11} \wedge \neg x_{10})$$
$$(\neg x_{12} \vee \neg x_{11}) \wedge (\neg x_{12} \vee \neg x_{10})$$
$$\wedge (x_{12} \vee x_{11} \vee x_{10})$$

Actual DIMACS clauses

```
-12 -11 0
-12 -10 0
12 11 10 0
```

This encoder reserves CNF variable 1 as a true constant. AIGER literals and DIMACS variable IDs therefore use different numbering.

The ALU's Actual I/O Port Maps

Selected LHS `design.iomap` rows

```
i a      0   2 +
i a      3   5 +
i op     0  10 +
o result 0  29 -
o cout   0 112 -
```

Selected RHS `design.iomap` rows

```
i a      0   2 +
i a      3   5 +
i op     0  10 +
o result 0  29 -
o cout   0 117 -
```

Each row records **direction, port, bit index, SAT ID, and polarity**. For example, LHS `result[0]` is $\neg x_{29}$.

The ALU's Actual I/O Port Maps

Selected LHS `design.iomap` rows

```
i a      0   2 +
i a      3   5 +
i op     0  10 +
o result 0  29 -
o cout   0 112 -
```

Selected RHS `design.iomap` rows

```
i a      0   2 +
i a      3   5 +
i op     0  10 +
o result 0  29 -
o cout   0 117 -
```

Each row records **direction, port, bit index, SAT ID, and polarity**. For example, LHS `result[0]` is $\neg x_{29}$.

Local IDs become distinct IDs in the miter

The LHS declares 112 variables, so every RHS ID is shifted by 112:

Port bit	LHS miter literal	RHS miter literal
`a[0]`	$+x_2$	$+x_{114}$
`result[0]`	$-x_{29}$	$-x_{141}$
`cout`	$-x_{112}$	$-x_{229}$

The ALU's Actual I/O Port Maps

Selected LHS `design.iomap` rows

```
i a      0   2 +
i a      3   5 +
i op     0  10 +
o result 0  29 -
o cout   0 112 -
```

Selected RHS `design.iomap` rows

```
i a      0   2 +
i a      3   5 +
i op     0  10 +
o result 0  29 -
o cout   0 117 -
```

Each row records **direction, port, bit index, SAT ID, and polarity**. For example, LHS `result[0]` is $\neg x_{29}$.

Local IDs become distinct IDs in the miter

The LHS declares 112 variables, so every RHS ID is shifted by 112:

Port bit	LHS miter literal	RHS miter literal
`a[0]`	$+x_2$	$+x_{114}$
`result[0]`	$-x_{29}$	$-x_{141}$
`cout`	$-x_{112}$	$-x_{229}$

Match by port name and bit; decode with the miter ID and polarity.

4. Build the Equivalence Miter

```
build/equivalence/miter_gen \
  "$W/lhs/design.cnf" "$W/lhs/design.iomap" \
  "$W/rhs/design.cnf" "$W/rhs/design.iomap" \
  "$W/miter.cnf"
```

The verification question added to the circuit constraints

Keep the two circuits' internal variables distinct.

Bind the 10 corresponding input bits with equality constraints.

Introduce 5 XOR variables for the corresponding output bits.

Require at least one XOR to be true.

4. Build the Equivalence Miter

```
build/equivalence/miter_gen \
  "$W/lhs/design.cnf" "$W/lhs/design.iomap" \
  "$W/rhs/design.cnf" "$W/rhs/design.iomap" \
  "$W/miter.cnf"
```

The verification question added to the circuit constraints

Keep the two circuits' internal variables distinct.

Bind the 10 corresponding input bits with equality constraints.

Introduce 5 XOR variables for the corresponding output bits.

Require at least one XOR to be true.

Generated miter header

```
p cnf 234 664
```

Does any common input make these two ALUs disagree?

5. Give the CNF to MiniSat

```
python3 solver.py "$W/miter.cnf" \
  --model "$W/model.txt" --solver minisat
```

Who sees what?

MiniSat receives the DIMACS CNF: variables and clauses.

It does not receive Verilog, AIGER, or signal names.

`solver.py` invokes MiniSat, validates a SAT assignment against the CNF, and writes a model for the decoder.

5. Give the CNF to MiniSat

```
python3 solver.py "$W/miter.cnf" \
  --model "$W/model.txt" --solver minisat
```

Who sees what?

MiniSat receives the DIMACS CNF: variables and clauses.

It does not receive Verilog, AIGER, or signal names.

`solver.py` invokes MiniSat, validates a SAT assignment against the CNF, and writes a model for the decoder.

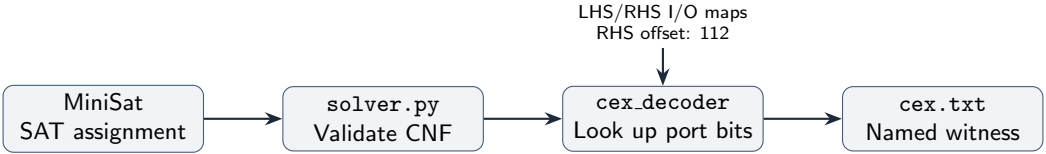
Captured result

`SATISFIABLE` (exit code 10)

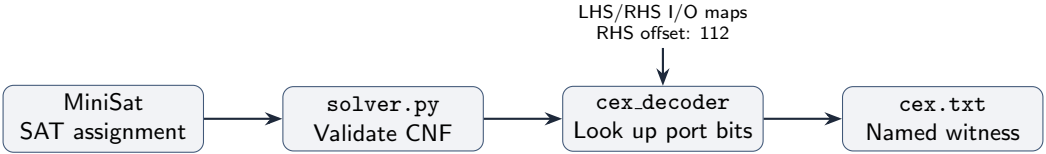
SAT means a mismatch exists. The model tells us which signal values witness it.

UNSAT (20) would establish equivalence; an error or timeout is inconclusive. The script handles SAT's nonzero exit code explicitly.

How the ALU SAT Model Becomes a Counterexample



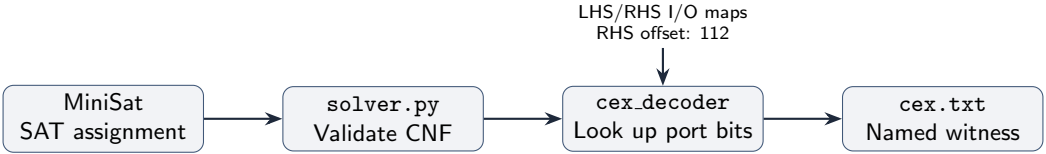
How the ALU SAT Model Becomes a Counterexample



Port	Mapped miter literals	Model variable values	Decoded bits
$a[3:0]$	(x_5, x_4, x_3, x_2)	$(1, 0, 1, 1)$	$1011_2 = 11$
$b[3:0]$	(x_9, x_8, x_7, x_6)	$(0, 1, 1, 1)$	$0111_2 = 7$
$op[1:0]$	(x_{11}, x_{10})	$(0, 0)$	00_2 (add)

Inputs: assemble bits from most significant to least significant. Input equalities ensure the RHS receives these same values.

How the ALU SAT Model Becomes a Counterexample



Port	Mapped miter literals	Model variable values	Decoded bits
$a[3:0]$	(x_5, x_4, x_3, x_2)	$(1, 0, 1, 1)$	$1011_2 = 11$
$b[3:0]$	(x_9, x_8, x_7, x_6)	$(0, 1, 1, 1)$	$0111_2 = 7$
$op[1:0]$	(x_{11}, x_{10})	$(0, 0)$	00_2 (add)
LHS result	$(-x_{102}, -x_{77}, -x_{52}, -x_{29})$	$(1, 1, 0, 1)$	$0010_2 = 2$
RHS result	$(-x_{214}, -x_{189}, -x_{164}, -x_{141})$	$(1, 1, 0, 1)$	$0010_2 = 2$
LHS carry	$-x_{112}$	$x_{112} = 1$	$\neg 1 = 0$
RHS carry	$-x_{229}$	$x_{229} = 0$	$\neg 0 = 1$

Inputs: assemble bits from most significant to least significant. Input equalities ensure the RHS receives these same values.

$a = 11$, $b = 7$, $op = 00$: the results agree, but carry differs. This is the decoded counterexample.

6. Decode and Explain the Counterexample

```
build/equivalence/cex_decoder "$W/model.txt" \
  "$W/lhs/design.cnf" "$W/lhs/design.iomap" \
  "$W/rhs/design.cnf" "$W/rhs/design.iomap"
```

Captured shared inputs

$a = 1011_2 = 11, \qquad b = 0111_2 = 7, \qquad op = 00_2$ (addition).

	LHS	RHS
`result`	0010_2	0010_2
`cout`	0	1

6. Decode and Explain the Counterexample

```
build/equivalence/cex_decoder "$W/model.txt" \
  "$W/lhs/design.cnf" "$W/lhs/design.iomap" \
  "$W/rhs/design.cnf" "$W/rhs/design.iomap"
```

Captured shared inputs

$a = 1011_2 = 11, \qquad b = 0111_2 = 7, \qquad op = 00_2$ (addition).

	LHS	RHS
`result`	0010_2	0010_2
`cout`	0	1

$11 + 7 = 18 = 10010_2$: both return the low four bits, 0010_2.

The faulty LHS takes bit 2 (0); the RHS takes carry bit 4 (1).

The ALUs are not equivalent. Finding the expected mismatch passes this test.

Reproduce the full flow with `make equiv TEST=alu N=4`. This is combinational equivalence, with no clock or BMC unrolling.

Measured Results: A Small Width Sweep

MiniSat; $N = 4, 16, 64$; one run per width; 20 s timeout per test command.

All 27 checks passed, including six BMC export checks; no timeouts.

Check	N	Result	Total (s)	Solver (s)
Faulty ALU	4	SAT	0.265	0.0028
Synthesis ALU	4	UNSAT	0.866	0.0020
Faulty ALU	16	SAT	0.315	0.0039
Synthesis ALU	16	UNSAT	0.767	0.0078
Faulty ALU	64	SAT	0.616	0.0080
Synthesis ALU	64	UNSAT	2.921	0.0643

Total: synthesis, encoding, solving, validation, and decoding.

Solver: subprocess wall time, including startup, parsing, preprocessing, search, and output. Tool compilation is excluded.

SAT is the expected success for the faulty ALU; UNSAT is expected for synthesis preservation.

11 Oct 2026; single-run workspace timings, not a scaling study. Timeouts are incomplete checks.

What the Result Proves

Combinational equivalence

SAT: a shared input makes at least one output differ; decode it as a counterexample.

UNSAT: the encoded circuits agree on every shared input, for all compared output bits.

Bounded model checking

SAT at depth k: a legal trace reaches a property failure at that depth.

UNSAT at every depth $0, \ldots, K$: no failure within that bound; this does not prove unbounded safety.

Current scope: gate-level JSON and ASCII AIGER; combinational equivalence and bounded sequential property checks. No sequential equivalence or unbounded proof.

A proof applies to the encoded model and query. A timeout or tool error is an incomplete check.